

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1.

In a basic particle system simulation using Python in Processing, each particle is represented as an object with its position, velocity, and a method to update its position based on its velocity. The following code attempts to create a single particle that moves across the screen but contains errors. Identify the correct way to fix the code.

1 / 1 point

```
1 class Particle:
2     def __init__(self, x, y):
3         self.position = PVector(x, y)
4         self.velocity = PVector(2, 3)
5
6     def update(self):
7         position.add(velocity)
8
9     def display(self):
10        ellipse(self.position.x, self.position.y, 10, 10)
11
12 def setup():
13     size(400, 400)
14     global p
15     p = Particle(200, 200)
16
17 def draw():
18     background(255)
19     p.update()
20     p.display()
21
```

- ☒

1 #replace:

2 position.add(velocity)

3 #with:

4 self.position.add(self.velocity)

5 #in the update method.
- ☐

1 #replace:

2 position.add(velocity)

3 #with:

4 PVector.add(self.position, self.velocity)

5 #in the update method.
- ☐

1 #replace:

2 position.add(velocity)

3 #with:

4 self.position = PVector.add(position, velocity)

5 #in the update method.
- ☐

1 #replace:

2 ellipse(self.position.x, self.position.y, 10, 10)

3 #with:

4 ellipse(position.x, position.y, 10, 10)

5 #in the display method.
- ☐

1 #replace:

2 p = Particle(200, 200)

3 #with:

4 p = new Particle(200, 200)

5 #in the setup method.

☒ Correct

Correct. This solution fixes the error by properly referencing the instance variables position and velocity using self, which is necessary within class methods to access or modify the object's attributes in Python.

2.

In a particle system using Python in Processing, each particle has a lifespan that decrements over time until it reaches zero, at which point the particle should "die" or cease to be updated or displayed. The following code attempts to implement a lifespan for each particle but contains errors. Identify the correct way to fix the code.

1 / 1 point

```
1 class Particle:
2     def __init__(self, x, y, lifespan):
3         self.position = PVector(x, y)
4         self.lifespan = lifespan # lifespan in frames
5
6     def update(self):
7         self.position.add(PVector(2, 3))
8         lifespan -= 1
9
10    def display(self):
11        stroke(0, self.lifespan)
12        fill(127, self.lifespan)
13        ellipse(self.position.x, self.position.y, 10, 10)
14
15    def isDead(self):
16        if lifespan < 0:
17            return True
18        else:
19            return False
20
21 def setup():
22     size(400, 400)
23     global p
24     p = Particle(200, 200, 255)
25
26 def draw():
27     background(255)
28     if not p.isDead():
29         p.update()
30         p.display()
31
```

- ☒

1 #replace

2 lifespan -= 1

3 #with

4 self.lifespan -= 1

5 #in the update method.

6

7

8

☒ Correct

Correct. This solution addresses the issue by correctly referring to the instance variable lifespan with self.lifespan, which is necessary to modify the object's lifespan.

- ☒

1 #replace

2 if lifespan < 0:

3 #with

4 if self.lifespan < 0:

5 #in the isDead method.

6

7

8

☒ Correct

Correct. This change correctly references the lifespan attribute of the instance by using self.lifespan, fixing the scope issue where lifespan is not recognized as an instance variable without self.

- ☐

1 #replace

2 stroke(0, self.lifespan)

3 #with

4 stroke(0)

5 #and

6 fill(127, self.lifespan)

7 #with

8 fill(127)

9 #in the display method.

10

11

12

- ☐

1 #in the display method, replace

2 ellipse(self.position.x, self.position.y, 10, 10)

3 #with

4 if self.lifespan > 0: ellipse(self.position.x, self.position.y, 10, 10).

5

6

7

- ☐

1 #replace

2 p = Particle(200, 200, 255)

3 #with

4 p = Particle(200, 200)

5 #in the setup method.

6

7

8

3.

In the context of a particle system in Processing using Python, you're tasked with implementing the displayAge function for a Particle class. This function should modify the particle's color based on its age, with younger particles appearing more vibrant and older particles fading to black. The particle's age is stored in the age attribute, and its maximum age before disappearing is stored in maxAge. The following skeleton code includes the displayAge function call but not its implementation. How should the displayAge function be written?

1 / 1 point

```
1 class Particle:
2     def __init__(self, x, y, maxAge):
3         self.position = PVector(x, y)
4         self.age = 0
5         self.maxAge = maxAge
6
7     def update(self):
8         self.position.add(PVector.random2D())
9         self.age += 1
10
11    def display(self):
12        ellipse(self.position.x, self.position.y, 10, 10)
13
14    def displayAge(self):
15        # Function implementation needed here
16
17 def setup():
18     size(400, 400)
19     global p
20     p = Particle(200, 200, 255)
21
22 def draw():
23     background(255)
24     p.update()
25     p.displayAge()
26
```

- ☐ Option 1:
- 1 def displayAge(self):

2 fill(255 - self.age, 0, 0)

3 ellipse(self.position.x, self.position.y, 10, 10)

4
- ☒ Option 2:
- 1 def displayAge(self):

2 fill(255 - self.age * (255 / self.maxAge), 255 - (self.age * (255 / self.maxAge)), 255 - (self.age * (255 / self.maxAge)))

3 ellipse(self.position.x, self.position.y, 10, 10)

4
- ☐ Option 3:
- 1 def displayAge(self):

2 fill(self.age, self.age, self.age)

3 ellipse(self.position.x, self.position.y, 10, 10)

4
- ☐ Option 4:
- 1 def displayAge(self):

2 lifespan = self.maxAge - self.age

3 fill(lifespan, lifespan, lifespan)

4 ellipse(self.position.x, self.position.y, 10, 10)

5

☒ Correct

Correct. This implementation properly fades the particle from white to black as it ages, accurately representing the particle's age across its lifespan. It linearly interpolates the color based on the current age and maximum age, applying the change uniformly across all color components.